

Documentación del Laboratorio 2: Implementación de un Sistema Empotrado RISC-V en FPGA

Jeshua Chavarría Chinchilla
Universidad de Costa Rica

Enero 2026

1 Introducción

El Laboratorio 2 de la asignatura **EL3313 Taller de Diseño Digital** tiene como objetivo la implementación de un sistema empujado en una FPGA Nexys4 basado en un microcontrolador RISC-V (RV32I). Este sistema incluye una memoria ROM para almacenar instrucciones, una memoria RAM para datos temporales, y periféricos como UART para la comunicación serial, LEDs para la indicación visual del estado del sistema, y switches/botones para la entrada de datos.

El sistema está diseñado para recibir operaciones aritméticas a través de UART (como suma o resta), procesarlas y devolver los resultados de manera eficiente. Este laboratorio combina conceptos de diseño de sistemas empujados, comunicación serie y programación en ensamblador para la arquitectura RISC-V.

2 Objetivos

El objetivo principal de este laboratorio es diseñar e implementar un sistema embebido utilizando un microcontrolador RISC-V sobre una FPGA. Los objetivos específicos incluyen:

- Implementar un microcontrolador basado en RISC-V RV32I.
- Establecer comunicación UART para recibir y enviar datos.
- Controlar periféricos como LEDs y switches.
- Utilizar memoria ROM para almacenar instrucciones y memoria RAM para almacenamiento de datos.

3 Descripción del Sistema

El sistema está compuesto por los siguientes módulos:

- **Microcontrolador RISC-V:** Basado en el subconjunto RV32I, implementado en SystemVerilog y sintetizado en la FPGA.
- **Memoria ROM:** Almacena las instrucciones del programa.
- **Memoria RAM:** Utilizada para almacenar datos temporales durante la ejecución.
- **Periféricos:**
 - *Switches/Botones:* Para entrada de datos, con anti-rebote.
 - *LEDs:* Para mostrar el estado del sistema.
 - *UART:* Para la comunicación serial con una laptop.

3.1 Diagrama de Bloques del Sistema

A continuación, se presenta un diagrama de bloques del sistema implementado:

4 Implementación del Sistema

El sistema fue implementado utilizando el núcleo PicoRV32, que es una implementación ligera de un microcontrolador RISC-V. Este núcleo fue integrado con otros módulos como el bus driver y los periféricos.

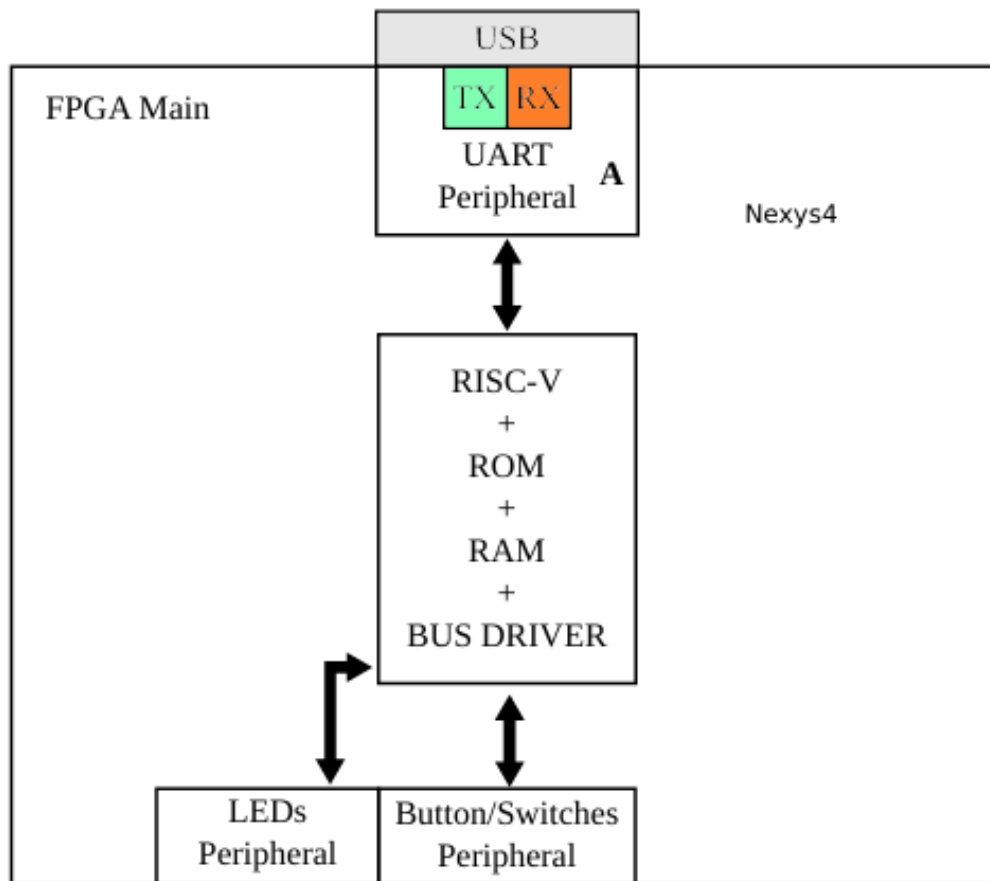


Figure 1: Diagrama de bloques del sistema RISC-V en FPGA.

4.1 Microcontrolador RISC-V

El microcontrolador implementa el subconjunto RV32I, que incluye instrucciones para operaciones aritméticas básicas y control de flujo. El núcleo ejecuta instrucciones almacenadas en la memoria ROM y accede a la memoria RAM para almacenar datos temporales.

4.2 Bus Driver

El `bus_driver` es un módulo que se encarga de la interconexión entre la memoria, los periféricos y el microcontrolador. Se encarga de gestionar las señales de control de lectura/escritura hacia la memoria RAM y los periféricos. El siguiente fragmento de código muestra la implementación básica del bus driver:

Listing 1: Fragmento de código de `bus_driver.sv`

```
module bus_driver (
    input logic      clk_i,
    input logic      rst_i,
    input logic [31:0] DataAddress_i,
    input logic [31:0] DataOut_i,
    input logic      we_i,
    output logic [14:0] ram_addr_o,
    output logic [31:0] ram_wdata_o,
    output logic [0:0] ram_we_o,
    input logic [31:0] ram_rdata_i,
    output logic [31:0] periph_addr_o,
    output logic [31:0] periph_wdata_o,
    output logic      periph_we_o,
    input logic [31:0] periph_rdata_i,
    output logic [31:0] DataIn_o
);
    // Implementacin de la lgica de decodificacin
    // de direccin y control de perifricos
    // ( Cdigo completo de decodificacin y seales
    // combinacionales)
endmodule
```

4.3 UART

El módulo UART se encarga de la transmisión y recepción de datos seriales a través de los pines de la FPGA. Utiliza el protocolo 8N1 (8 bits de datos, sin paridad, 1 bit de parada). A continuación, se muestra un fragmento del código de la transmisión UART:

Listing 2: Fragmento de código de `uart_tx.sv`

```
module uart_tx #(
    parameter int CLKS_PER_BIT = 10417
)(
    input logic      i_Clock,
    input logic      i_Tx_DV,    // Seal de datos
    // vltidos
    input logic [7:0] i_Tx_Byte,
    output logic      o_Tx_Active,
    output logic      o_Tx_Serial,
```

```
    output logic      o_Tx_Done
);
    // Implementacin de la lgica de transmisin
    UART
    // ( Cdigo completo de la transmisin de datos)
endmodule
```

5 Pruebas

Se implementaron testbenches en SystemVerilog para verificar el funcionamiento correcto del sistema. Los test incluyen la escritura y lectura de la memoria RAM, así como la comunicación UART.

5.1 Descripción de los testbenches

Para validar el funcionamiento del sistema se desarrollaron varios testbenches en SystemVerilog. Cada prueba fue diseñada para verificar módulos específicos del sistema y confirmar que la comunicación entre componentes funcionara correctamente antes de la implementación final en FPGA.

5.1.1 tb_bus_driver.sv

Este testbench verifica el funcionamiento del módulo `bus_driver`. Su objetivo principal es comprobar que la decodificación del mapa de memoria se realice correctamente, diferenciando entre accesos a RAM y accesos a periféricos.

En esta prueba se simulan escrituras y lecturas en la RAM, por ejemplo en las direcciones `0x40010` y `0x40020`. También se verifica la escritura al registro de LEDs en la dirección `0x02004` y la lectura del registro de switches en `0x02000`.

Se espera que:

- Los datos escritos en RAM puedan recuperarse correctamente.
- Las escrituras a periféricos modifiquen los registros correspondientes.
- Las lecturas de periféricos devuelvan los valores esperados.
- La latencia de un ciclo del sistema se mantenga correctamente.

5.1.2 tb_core_rom.sv

Este testbench valida la conexión entre el núcleo RISC-V y la memoria ROM del programa. La prueba instancia el módulo `riscv_core_wrapper` junto con la ROM inicializada con el archivo `programa.coe`.

Durante la simulación se monitorean las señales `prog_addr` y `prog_data`, permitiendo observar el flujo de instrucciones ejecutadas por el procesador.

Se espera que:

- El procesador inicie en la dirección `0x0000`.
- Las instrucciones sean leídas correctamente desde la ROM.
- El contador de programa avance de forma adecuada durante la ejecución.

5.1.3 `tb_core_ram.sv`

Este testbench verifica la integración completa entre el núcleo RISC-V, la ROM, la RAM y el módulo `bus_driver`. En esta prueba se analiza tanto la ejecución de instrucciones como el acceso a memoria de datos.

La simulación monitorea señales como `prog_addr`, `data_addr`, `data_we`, `data_out` y `data_in`, permitiendo observar el comportamiento general del sistema.

Se espera que:

- El núcleo ejecute instrucciones correctamente desde la ROM.
- Las operaciones de lectura y escritura hacia RAM se realicen correctamente.
- Las señales de control del bus se activen en los momentos adecuados.
- El sistema mantenga sincronización correcta entre memoria y procesador.

5.2 Test de Memoria RAM

Se realizaron pruebas para verificar que los datos pueden ser escritos y leídos correctamente desde la memoria RAM. A continuación se muestra un test de escritura y lectura en la dirección `0x40010`:

```
initial begin
    $display("=== Test de Memoria RAM ===");
    write32(32'h40010, 32'h1234_5678);
    read32(32'h40010, rd);
    if (rd == 32'h1234_5678)
        $display("PASS: 0x%08h", rd);
    else
        $display("FAIL: 0x%08h (esperado 0x12345678)", rd);
end
```

Figure 2: Fragmento de código de `bus_driver.sv`.

6 Conclusiones

Este laboratorio permitió la implementación de un sistema embotado basado en un microcontrolador RISC-V en una FPGA Nexys4. Se lograron integrar periféricos como UART, LEDs y switches para controlar el sistema. Las pruebas realizadas confirmaron que el sistema funciona correctamente y es capaz de realizar operaciones aritméticas a través de UART.

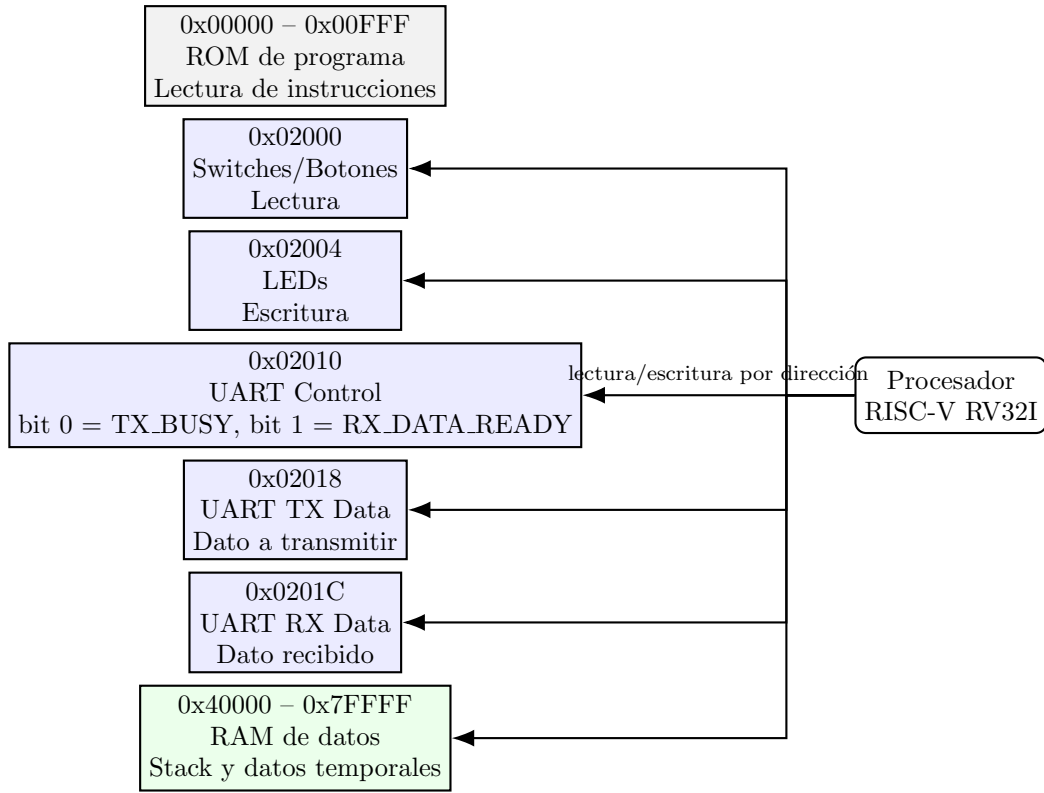


Figure 3: Mapa de memoria utilizado por el sistema.

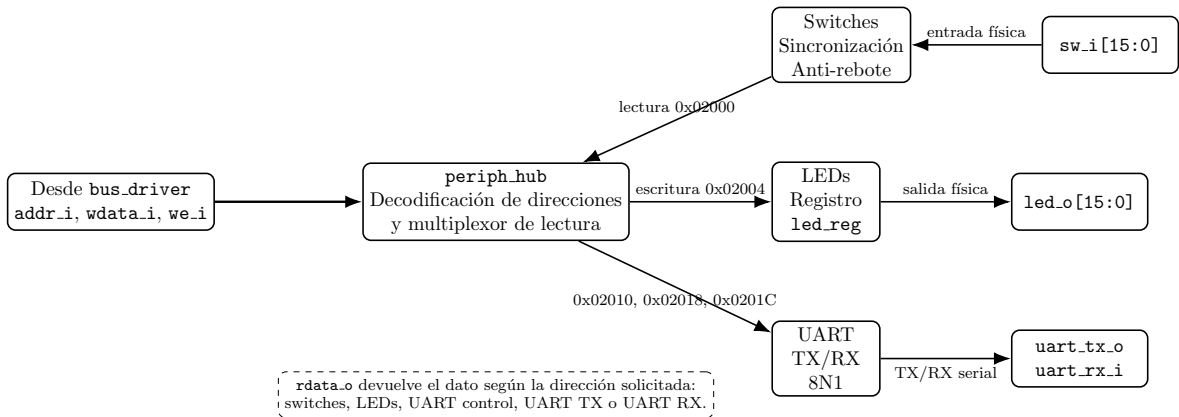


Figure 4: Organización interna del módulo periph_hub.